Trabajo práctico final

Comunicación inalámbrica - Uso de SPI y UART via DMA

Matías Morales Gariglio

19 de abril de 2026

1. Link repositorio

El código fuente y la documentación del proyecto se encuentran disponibles en: repositorio oficial de GitHub.

2. Descripción General del Sistema

El sistema implementado consiste en un enlace de comunicación inalámbrica, el mismo será la base para el trabajo práctico final de la materia (gestión inteligente de recursos hídricos en entornos domésticos o industriales). Utilizando el transceptor **nRF24L01** en la banda ISM de 2.4 GHz, se establece una arquitectura de red inalámbrica para la adquisición y procesamiento de datos en tiempo real.

El proyecto se centra en la creación de un nodo de telemetría remoto (Slave) encargado de los sensores y un nodo central (Master) que procesa la información y gestiona las interfaces de salida. El sensor principal será el HC-SR04 (sensor de proximidad), cuya información será procesada y transmitida por el slave inalámbricamente. El master recibirá esta información y la mostrará de 3 formas distintas:

- Display luminoso led, tipo barras verticales
- UART en PC
- Alerta sonora cuando se llegue a un nivel mínimo

2.1. Arquitectura de archivos

Los archivos se acomodaron en la carpeta API, aquellos que corresponden exclusivamente tanto a maestro como a esclavo. Creé una carpeta denominada Personal_libs para portabilidad del código. Lo que hice en ports_master.h y ports_slave.h es cortar lo generado en stmCubeMx (defines de puertos y similares) del main.h y colocarlo allí. Lo malo de esto es que cualquier modificación futura en stmCubeMx generará nuevamente el main.h y podría generar un conflicto entre definiciones.

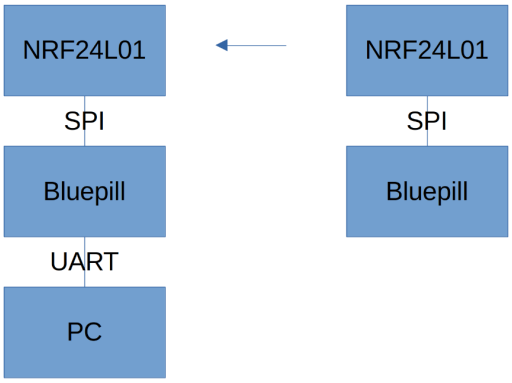

Figura 1: Diagrama en bloques del sistema propuesto.

```
---
MASTER (STM32F103C8T6)

Master_0/Master_0/
├── Core/
│   ├── Inc/
│   │   ├── main.h              ← generado CubeMX (pines + HAL)
│   │   ├── dma.h / gpio.h / spi.h / usart.h
│   │   └── stm32f1xx_it.h
│   └── Src/
│       ├── main.c
│       ├── dma.c / gpio.c / spi.c / usart.c
│       └── stm32f1xx_it.c
└── STM32CubeIDE/Drivers/API/
    ├── Inc/
    │   ├── ports_master.h      ← pines manuales + HAL (portabilidad)
    │   ├── API_display.h       ← display 74HC595 + estados
    │   └── API_parser.h        ← FSM recepción NRF + UART
    └── Src/
        ├── API_display.c
        └── API_parser.c
```

Figura 2: Archivos en Master

```
SLAVE_0 (STM32F103C8T6)

SLAVE_0/Slave_0/
├── Core/
│   ├── Inc/
│   │   ├── main.h              ← generado CubeMX
│   │   ├── dma.h / gpio.h / spi.h / usart.h / tim.h / adc.h
│   │   └── stm32f1xx_it.h
│   └── Src/
│       ├── main.c
│       ├── dma.c / gpio.c / spi.c / usart.c / tim.c / adc.c
│       └── stm32f1xx_it.c
└── STM32CubeIDE/Drivers/API/
    ├── Inc/
    │   ├── ports_slave.h       ← pines manuales + HAL (portabilidad)
    │   ├── API_parser.h        ← FSM medición JSN + transmisión NRF
    │   └── STM_init.h
    └── Src/
        └── API_parser.c
```

Figura 3: Archivos en slave

```
Personal_libs (compartidas MASTER y SLAVE)

Personal_libs/
├── NRF24L01/
│   ├── Inc/nrf24l01.h
│   └── Src/nrf24l01.c          ← driver SPI+DMA, TX/RX
├── JSNSR04T/
│   ├── Inc/jsnsr04t.h
│   └── Src/jsnsr04t.c          ← driver TIM2 Input Capture
├── Delay/
│   ├── Inc/u_delay.h
│   └── Src/u_delay.c           ← delay no bloqueante DWT
├── UART_MX/
│   ├── Inc/uart_mx.h
│   └── Src/uart_mx.c           ← wrapper UART CubeMX
└── UART/
    ├── Inc/API_uart.h
    └── Src/API_uart.c
```

Figura 4: Archivos en común, visibles entre maestro y esclavo

3. Delay

El módulo JSNSR04T requiere un pulso de 10 uS, por ello tuve que generar un nuevo tipo de delay.

La implementación de las funciones de temporización se encuentra en el archivo: u_delay.c (línea 17).

Por ello utilicé otro enfoque. DWT: Data watchpoint and trace. Habilitando el bit TRCENA (trace enable) en el registro de control de monitoreo y depuración del micro, habilita un contador de ciclos DWT. Esto lo que termina haciendo es contar la cantidad de ciclos de reloj por los que pasó el microcontrolador. De esta forma, si el microcontrolador está seteado con el PLL para trabajar a una frecuencia de 72 MHz, cada 72 flancos de clock tendremos un micro segundo (Esto se ve en la línea SystemCoreClock / 1000000U). La lógica de funcionamiento es idéntica a la implementada en clases, con start, running y duration.

4. Transceptor nRF24L01

El **nRF24L01** es un transceptor monolítico de radiofrecuencia (RF) que opera en la banda ISM (*Industrial, Scientific, and Medical*) de 2.4 GHz. Diseñado por *Nordic Semiconductor*, este módulo destaca por su bajísimo consumo energético y su motor de protocolos acelerado por hardware, denominado **Enhanced ShockBurst™**.

4.1. Características Principales

Banda de Operación: 2.400 a 2.525 GHz (125 canales seleccionables).

Velocidad de Transmisión: Configurable entre 250 kbps, 1 Mbps y 2 Mbps.

Interfaz de Comunicación: SPI (*Serial Peripheral Interface*) de hasta 10 Mbps.

Consumo de Energía: Extremadamente bajo (aprox. 11.3 mA en transmisión a 0 dBm y 13.5 mA en recepción a 2 Mbps).

Tolerancia de Voltaje: Alimentación de 1.9 a 3.6 V, con pines de entrada tolerantes a 5 V, facilitando la interfaz con diversas familias de microcontroladores.

4.2. Ventajas para el Proyecto

- **Gestión de Paquetes Automática:** El hardware gestiona el preámbulo, la dirección y la verificación de redundancia cíclica (CRC), reduciendo la carga computacional del microcontrolador STM32.
- **MultiCeiver™:** Capacidad de recibir datos de hasta seis nodos diferentes simultáneamente en topología estrella, lo cual simplifica la expansión futura del ecosistema domótico.
- **Dimensiones Reducidas:** Su factor de forma compacto permite una integración eficiente en el diseño de los PCB personalizados realizados en KiCad.

5. Sensor de Ultrasonido Impermeable JSN-SR04T

El **JSN-SR04T** es un transductor ultrasónico de distancia diseñado para aplicaciones que requieren resistencia a condiciones ambientales adversas. A diferencia de los sensores piezoeléctricos convencionales, este dispositivo cuenta con un transductor estanco y un cable de extensión (típicamente de 2.5 metros), lo que permite separar la etapa de procesamiento electrónico del punto de medición.

5.1. Especificaciones Técnicas

El sensor presenta un rendimiento robusto bajo las siguientes características nominales:

Cuadro 1: Parámetros operativos del JSN-SR04T.

Parámetro	**Valor Nominal**
Tensión de operación	3.0 V a 5.5 V DC
Corriente de trabajo	< 30 mA
Frecuencia acústica	40 kHz
Rango de medición	20 cm a 600 cm
Resolución	1 cm
Ángulo de apertura	$< 75°$

5.2. Principio de Funcionamiento

El dispositivo opera bajo el principio de *Time of Flight* (ToF), calculando la distancia en función del tiempo transcurrido entre la emisión de una ráfaga sónica y la recepción de su eco. El ciclo de medición se describe mediante la siguiente secuencia:

1. **Disparo (Trigger):** Se aplica un pulso de nivel alto (*High*) de al menos 10 μs en el pin `TRIG`.
2. **Emisión:** El sensor emite automáticamente una ráfaga de ocho pulsos a 40 kHz.

3. **Respuesta (Echo):** El pin `ECHO` se activa en nivel alto. La duración de este pulso (Δt) es proporcional a la distancia recorrida por la onda.

La distancia final se determina mediante la ecuación cinemática, considerando la velocidad del sonido ($v \approx 0.0343$ cm/us):

$$d = \frac{\Delta t \times v}{2} \tag{1}$$

6. Periféricos utilizados

6.1. Interfaz SPI

Ambos nodos del sistema —Slave_0 y Master_0— utilizan el periférico `SPI1` del microcontrolador STM32F103C8T6 para comunicarse con el módulo de radio nRF24L01+. El bus opera en modo *Full-Duplex Master*, con el STM32 como maestro y el nRF como esclavo único. La selección del esclavo se realiza por software mediante el pin `CSN` (chip select negado), controlado manualmente desde el driver.

El nRF24L01+ requiere adicionalmente un pin `CE` (*Chip Enable*) para controlar los estados operativos de transmisión y recepción. Este pin no forma parte del bus SPI sino que es una señal de control independiente manejada por GPIO.

6.2. Configuración hardware

Los pines asignados al bus SPI1 son idénticos en ambos nodos:

Pin	Señal	Dirección	Función
PA3	NRF_CE	Salida	Control de modo TX/RX
PA4	NRF_CSN	Salida	Chip Select (activo en bajo)
PA5	SPI1_SCK	Salida	Reloj SPI
PA6	SPI1_MISO	Entrada	Datos maestro ← esclavo
PA7	SPI1_MOSI	Salida	Datos maestro → esclavo
PB0	NRF_IRQ	Entrada	Interrupción (EXTI, flanco descendente)

Cuadro 2: Pinout SPI y control nRF24L01+ (idéntico en ambos nodos)

Los parámetros del periférico configurados en STM32CubeMX son:

- **Modo:** Full-Duplex Master
- **Polaridad de reloj (CPOL):** 0 — reposo en bajo
- **Fase de reloj (CPHA):** 0 — muestreo en flanco ascendente (Modo 0)
- **Prescaler:** /8 → frecuencia de reloj: $\frac{72\,\text{MHz}}{8} = 9\,\text{MHz}$
- **NSS:** software (control manual por GPIO)
- **DMA:** DMA1 canal 2 (SPI1_RX), DMA1 canal 3 (SPI1_TX)

6.3. Driver nRF24L01+

El driver fue desarrollado como biblioteca personal compartida entre ambos nodos, ubicada en `Personal_libs/NRF24L01/`. Toda comunicación con el módulo se realiza a través de `HAL_SPI_TransmitReceive_DMA`, aprovechando el modo full-duplex: cada transacción envía un comando por MOSI y recibe simultáneamente la respuesta por MISO.

6.3.1. Sincronización con DMA

La transferencia es no bloqueante: el DMA mueve los datos de forma autónoma y, al completarse, dispara la interrupción que ejecuta el callback:

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi) {
    csnHigh();
    _busy = false;
}
```

El flag `_busy` (declarado `volatile`) sincroniza el estado del bus entre el hilo principal y el contexto de interrupción. Para evitar bloqueos ante fallas del DMA —situación detectada durante el desarrollo donde `HAL_SPI_TransmitReceive_DMA` devolvía `HAL_BUSY` ocasionalmente—, se implementó la función `_waitBusy()` con timeout basado en el contador DWT:

```
static void _waitBusy(void) {
    delayUsSet(&del_TIMEOUT, TIMEOUT);    /* TIMEOUT = 100 µs */
    while (_busy && !delayUsRead(&del_TIMEOUT)) {}
    if (_busy) {
        _busy = false;
        csnHigh();
    }
}
```

Si el DMA no libera el bus en 100 µs, el driver recupera el estado manualmente elevando `CSN` y limpiando el flag. La idea fue implementar una suerte de watchdog para el periférico.

Observación: Si bien se observó una mejoría de la estabilidad del sistema con esta implementación, actualmente estoy lidiando con un problema en los módulos. Al iniciar el slave, algunas veces no transmite información y el sistema 'cuelga'. Hay que reiniciar manualmente para que inicie. Sospecho de una fuente de alimentación inestable (3v3 generados con un ams1117, con capacitores de 100 nF SMD y 470 uF electrolíticos). El próximo paso a probar es implementar un módulo step down para ver de solucionar esto.

6.3.2. Funciones de acceso al bus

- `nrfWriteReg(reg, val)`: escribe un registro de 8 bits. Envía el comando `W_REGISTER` (0x20 | reg) seguido del valor.

- `nrfReadReg(reg)`: lee un registro. Envía `R_REGISTER` (0x00 | reg) y `NOP` (0xFF); el dato útil está en `_rx_buf[1]`.
- `nrfWriteAddr(reg, addr, len)`: escribe una dirección de hasta 5 bytes en un ciclo DMA único.
- `nrfTransmit(data, len)`: carga el payload en la TX FIFO mediante el comando `W_TX_PAYLOAD` (0xA0), espera a que el DMA finalice y genera el pulso de `CE` ($\geq$10 µs mínimo según datasheet, implementado con `HAL_Delay(1)`).
- `nrfRecept(data, len)`: verifica el flag `RX_DR` en el registro `STATUS`; si hay dato disponible, lo lee con el comando `R_RX_PAYLOAD` (0x61) y limpia el flag.

6.4. Configuración del módulo nRF24L01+

Ambos nodos utilizan los mismos parámetros de radio. La tabla 3 resume la configuración de registros aplicada en la inicialización.

Registro	**Dir.**	**Valor**	**Configuración**
CONFIG	0x00	0x0E / 0x0F	PWR_UP=1, CRC habilitado (1 byte); PRIM_RX=0 (TX) / PRIM_RX=1 (RX)
EN_AA	0x01	0x00	Auto-acknowledge deshabilitado en todos los pipes
SETUP_RETR	0x04	0x00	Sin retransmisiones (ARC=0)
RF_CH	0x05	0x64 (100)	Canal 100 → 2500 MHz
RF_SETUP	0x06	0x06	1 Mbps, potencia máxima (0 dBm)
RX_PW_P0	0x11	10	Payload fijo de 10 bytes (`sizeof(payload_t)`)
TX_ADDR / RX_ADDR_P0	0x10/0x0A	{`0xEF, 0xEF, 0xEF, 0xEF, 0xEF`} (5 bytes)	

Cuadro 3: Registros del nRF24L01+ configurados en la inicialización

Por lo pronto, el sistema opera en modo *fire-and-forget*: el transmisor no espera confirmación de recepción. Esta decisión simplifica el protocolo a costa de no garantizar la entrega de cada paquete individual.

6.5. Estructura del payload

La estructura de datos transmitida por SPI desde el Slave hacia el módulo nRF —y recibida por el Master— ocupa exactamente 10 bytes:

```
typedef struct {
    uint16_t nivel;        /* distancia en cm              */
    uint16_t caudal;       /* pulsos por segundo           */
    uint16_t luminosidad;  /* valor ADC raw (12 bits)      */
    int16_t  temperatura;  /* temperatura en °C × 10       */
    uint8_t  humedad;      /* humedad relativa en %        */
} payload_t;
```

Esto se diseñó de esta forma para una posible modificación a futuro, teniendo ya prevista la trama en el caso de que se desee implementar algún otro tipo de sensor.

Observación: El padding al final existe para que, al ponerse dos payload_t seguidos en un array, el segundo también empiece en dirección par:

```
payload\_t arr[2];
// &arr[0] = 0x00  // par
// &arr[1] = 0x0A  // par  (sin padding sería 0x09, impar -> problema)
```

Por eso sizeof(payload_t) == 10 y no 9.

6.6. Uso del SPI en cada nodo

Slave_0 (transmisor). Inicializa el módulo en modo TX mediante `nrfSetModeTX()`. En cada ciclo de la FSM, una vez obtenida la medición del sensor JSN-SR04T, invoca `nrfTransmit()` para enviar la estructura `payload_t` al Master. La FSM transita por los estados: `MEASURE` → `MEASURING` → `UART` → `NRF` → `WAITING` → `MEASURE`.

Master_0 (receptor). Inicializa el módulo en modo RX mediante `nrfSetModeRX()`, con `CE` en alto de forma permanente. En cada ciclo de la FSM consulta `nrfRecept()` para verificar si hay un paquete disponible en la RX FIFO. Ante 10 segundos sin recepción, un watchdog por software pone `payload.nivel = 0` y actualiza el display. La FSM transita por: `RECEIVE` → `UART` → `PRINT_LEV` → `RECEIVE`.

Adicionalmente, al finalizar `initParser()`, el Master verifica que el registro `CONFIG` del nRF valga `0x0F`; en caso contrario ejecuta `NVIC_SystemReset()` para forzar una reinicialización completa del sistema.

6.7. Interfaz UART

Ambos nodos utilizan `USART1` del STM32F103C8T6 exclusivamente como canal de telemetría hacia una terminal serie en PC. El periférico opera en modo transmisión únicamente (*TX-only*); la recepción está implementada en el driver pero no se usa en la aplicación del TPF.

6.8. Configuración hardware

Pin	Señal	Dirección	Función
PA9	USART1_TX	Salida	Transmisión de datos hacia PC
PA10	USART1_RX	Entrada	Recepción (configurado, no utilizado)

Cuadro 4: Pinout UART1 (idéntico en ambos nodos)

Parámetros configurados en STM32CubeMX:

- **Baud rate:** 115200 bps
- **Trama:** 8 bits de datos, sin paridad, 1 bit de stop (8N1)

- **DMA TX:** DMA1 canal 4 (`USART1_TX`)

Observación: El GPIO en TX RX se encuentra protegido con un resistor de 330 ohms para proteger al gpio ante un corto circuito accidental. El handler para UART se implementó al setear STMCubeMx, por lo que no fue necesario realizar la configuración hecha en clases.

6.9. Driver UART_MX

El driver fue desarrollado como biblioteca personal compartida entre ambos nodos, ubicada en `Personal_libs/UART_MX/`. Abstrae el `UART_HandleTypeDef` generado por CubeMX y expone una API de alto nivel independiente del periférico concreto.

El header incluye guards de compilación condicional para soportar tanto la familia STM32F1 como STM32F4:

```
#if defined(STM32F446xx)
    #include "stm32f4xx_hal.h"
#elif defined(STM32F103xB)
    #include "stm32f1xx_hal.h"
#endif
```

6.9.1. Inicialización

```
bool uartInit(UART_HandleTypeDef *uart);
```

Almacena el puntero al handle en una variable estática de módulo y inicializa el timer de timeout interno. Devuelve `false` si el puntero es `NULL`.

6.9.2. Transmisión

```
void uartSendString(uint8_t *pstring);
```

Espera a que el periférico quede en estado `HAL_UART_STATE_READY` antes de lanzar la transferencia mediante `HAL_UART_Transmit_DMA()`. Para evitar bloqueos indefinidos ante fallas del bus, la espera tiene un timeout de 10 ms implementado con el contador DWT:

```
delayUsSet(&del_uart, TIMEOUT);   /* TIMEOUT = 10000 µs = 10 ms */
while (UartHandle->gState != HAL_UART_STATE_READY
       && !delayUsRead(&del_uart));
HAL_UART_Transmit_DMA(UartHandle, pstring, strlen((char*)pstring));
```

Esto también busca de funcionar como un watch dog.

La longitud de la cadena se calcula en tiempo de ejecución con `strlen()`, por lo que la cadena debe estar terminada en `'\0'`.

6.9.3. Recepción

```
int8_t uartReceiveString(uint8_t *pstring);
```

Lee caracteres de a uno usando `HAL_UART_Receive()` con timeout de 1 ms por carácter, acumulando en un buffer estático interno. Retorna `1` cuando detecta `'\n'` o `'\r'`, `0` si no llegó ningún carácter en el timeout, y `-1` ante overflow (límite: `CMD_MAX_LINE = 64` bytes). Esta función no se utiliza en la aplicación actual del TPF.

6.10. Uso en cada nodo

Slave_0. Transmite una línea por cada medición del sensor JSN-SR04T, en el estado `UART` de la FSM:

```
snprintf(buf, sizeof(buf), "Medicion: %u cm \r\n", med);
uartSendString(buf);
```

Master_0. Transmite el valor recibido por radio en el estado `UART` de la FSM, junto con mensajes de diagnóstico al arranque (`''Sistema online''` y volcado de registros del nRF):

```
snprintf(buff, sizeof(buff), "\nMedicion: %u \r", payload.nivel);
uartSendString(buff);
```

En ambos casos el buffer de transmisión es local a la FSM (32 bytes), formateado con `snprintf()` antes de cada envío.

7. Arquitectura del hardware

El sistema se divide en dos unidades funcionales con las siguientes características:
Nodo transmisor:

- **Unidad de Procesamiento:** Microcontrolador STM32F103C8T6 (Blue Pill).
- **Interfaz de Radio:** Módulo nRF24L01 conectado mediante bus SPI.
- **Sensores:** Entrada para sensor ultrasónico JSN-SR04T (impermeable) para medición de nivel/distancia.

Nodo receptor

- **Unidad de Procesamiento:** Microcontrolador STM32F103C8T6 (Blue Pill).
- **Interfaz de Radio:** Módulo nRF24L01 (SPI).
- **Interfaz de Usuario/PC:** Comunicación via UART hacia una PC para la visualización de datos mediante un terminal serie o interfaz gráfica.

- **Indicadores luminosos y sonoros:** Implementación de un display de segmentos verticales para indicar nivel y buzzer de alarma sonora.

Se adjuntan los esquemáticos del hardware implementado en la carpeta del trabajo final, tanto del master como del slave. Como punto a destacar, se observa el uso de resistores a la salida del gpio para proteger las salidas. En los casos de las señales a 5v, se utilizó un zenner de 3v3 como adaptador de tensión para el gpio. En el master, se observa el uso de un registro de desplazamiento como driver para los leds, mediante el uso de un 74hc595.

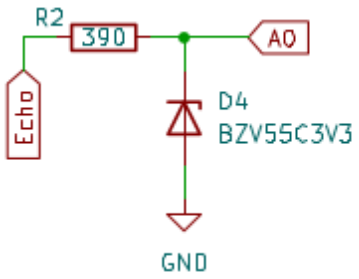

Figura 5: Protecciones y adaptaciones de voltajes para el GPIO.

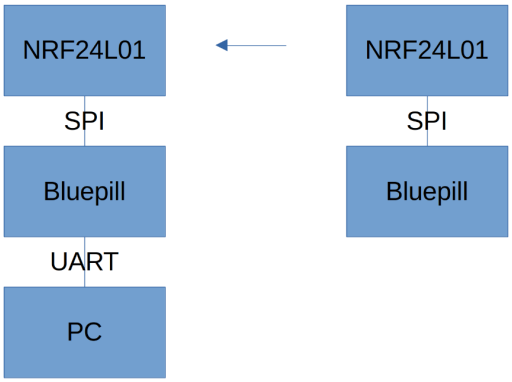

Figura 6: Diagrama en bloques del sistema propuesto.

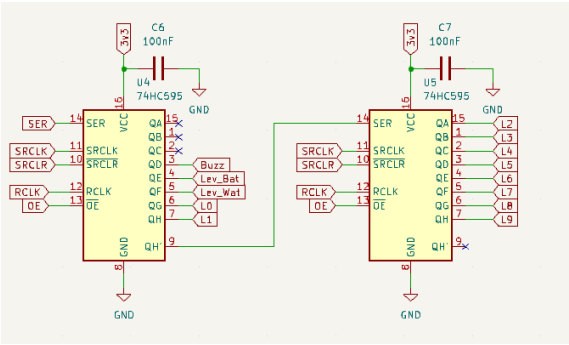

Figura 7: Interconexión entre registros de desplazamiento.

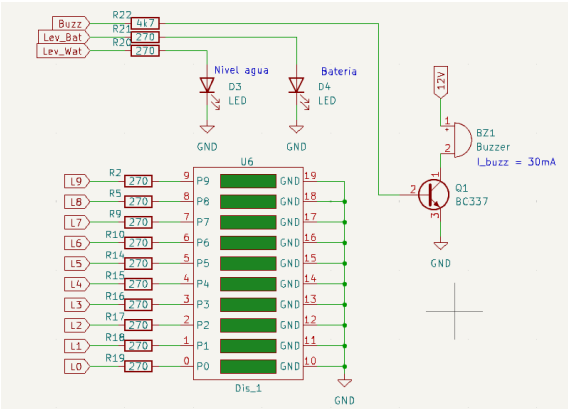

Figura 8: Sistema display implementado.

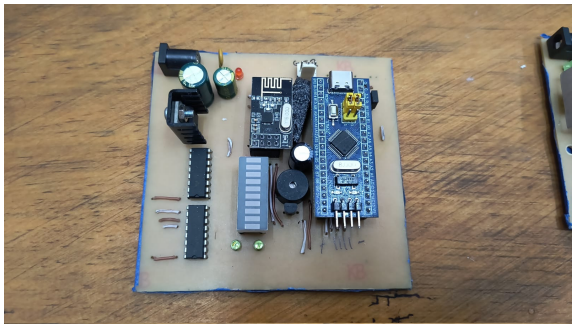

Figura 9: Circuito Master.

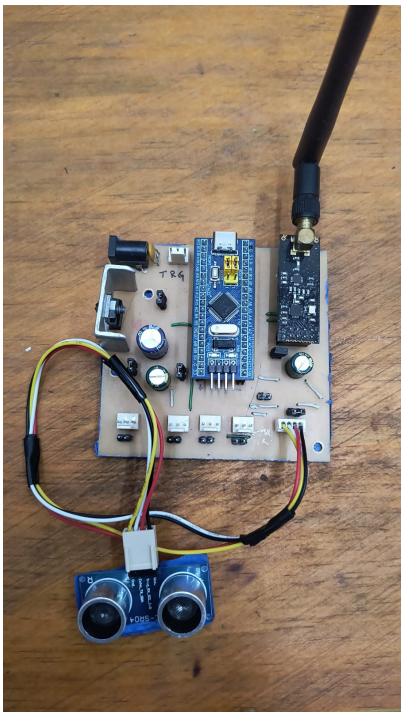

Figura 10: Circuito Slave.

La lógica de control de la pantalla se encuentra definida en el archivo API_display.c (línea 124). Se implementó un código similar a la función shiftOut(), conocida de Arduino. En resumen, lo que hace es transformar una trama de datos en una trama sincrónica.

8. Máquinas de estado

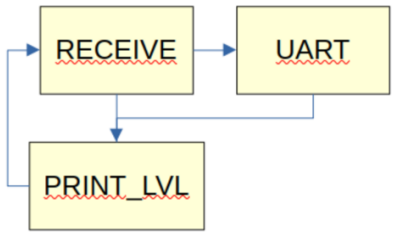

Figura 11: Máquina de estados del master.

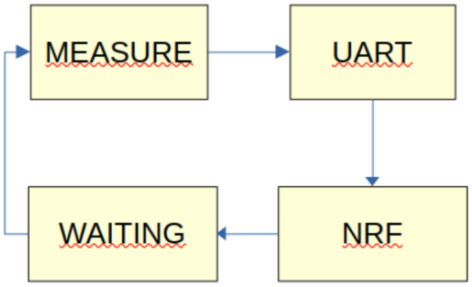

Figura 12: Máquina de estados del slave.

9. Ejemplo de variables globales privadas

Un ejemplo claro de este uso lo vemos en la variable de las máquinas de estado:
Variable global estática FMS - Slave

10. Actualización FSM - Master y slave

Se adjuntan las primeras líneas de actualización de las máquinas de estado, tanto del master como del slave.
Update FSM slave
Update FSM Master

11. The power of 10 rules

- **Sin goto ni recursiones:** Cumple.
- **Bucles con límites fijos:** Todos los while tienen salida por timeout DWT.
- **Sin memoria dinámica:** No hay malloc, calloc ni free en ningún módulo.
- **Funciones de una página:** Cumpel en general.
- **Mínima visibilidad:** Cumple. Todas las variables son static.
- **Preprocesador moderado:** Cumple. Solo se usa para constantes.
- **Punteros con una sola referencia:** Cumple. No hay punteros a punteros ni cadenas de desreferencias (-¿de un solo nivel).
- **Sin punteros a funciones:** Cumple. No hay punteros a funciones en ningún módulo.

12. Ejemplos comentarios en líneas

Línea explicativa
Línea explicativa
Línea explicativa

12.1. Escalabilidad y Visión de Futuro

El diseño del sistema se ha concebido bajo un paradigma de **arquitectura modular**, permitiendo su integración como un nodo periférico dentro de un ecosistema de domótica más robusto.

Al emplear el estándar de comunicaciones nRF24L01+, el sistema queda preparado para:

- **Topología de Red:** Evolucionar de un enlace punto a punto hacia una red tipo estrella o malla (Mesh), integrando múltiples nodos sensores distribuidos en una vivienda o planta industrial.
- **Interoperabilidad:** Actuar como un *gateway* inalámbrico que, mediante la interfaz UART de la unidad central, pueda conectarse a plataformas de control de nivel superior como *Home Assistant*, *OpenHAB* o servidores MQTT sobre una Raspberry Pi.
- **Modularidad de Hardware:** La separación de las etapas de potencia (control de cargas como el lavarropas) y las etapas de lógica asegura que el módulo pueda adaptarse a diferentes aplicaciones domésticas sin requerir un rediseño del núcleo de procesamiento.